

Servidor de Armazenamento Local

Transformando um computador comum em um servidor de arquivos
acessível pelo navegador

Python · Flask · Cloudflare Tunnel · PowerShell · NSSM · SMTP

Documentação técnica e didática do projeto
Curso de Gestão da Tecnologia da Informação

Sumário

1. Sobre o projeto	3
2. Tecnologias utilizadas	4
3. Arquitetura do projeto	8
4. Aplicação Flask	10
5. Sistema de arquivos	12
6. Segurança dos caminhos	13
7. Cloudflare Tunnel	14
8. Script trycloudflared.ps1	16
9. Script enviar_email.py	18
10. Arquivo .env	20
11. NSSM	21
12. Inicialização completa do sistema	23
13. Exemplo de uso	24
14. Vantagens da solução	25
15. Limitações	26
16. Segurança e boas práticas	27
17. Estrutura do projeto	28
18. Problemas encontrados durante o desenvolvimento	29
19. Possíveis melhorias futuras	30
20. Conclusão	31

Antes de começar: Todos os exemplos de e-mail, senha, caminho de pasta e link usados neste documento são fictícios. Os blocos de código que aparecem aqui são versões simplificadas, escritas para explicar a lógica de cada parte. Eles não são cópias dos arquivos reais do projeto.

1. Sobre o projeto

Este projeto transforma um computador comum em um servidor de armazenamento de arquivos. A ideia é simples: o computador tem um disco com espaço sobrando, então esse espaço pode ser usado para guardar arquivos e permitir que eles sejam acessados de outro lugar, usando só o navegador.

Em vez de pagar por um serviço de armazenamento na nuvem, os arquivos ficam guardados **localmente**, no próprio computador. O computador funciona como servidor: ele fica ligado, executando a aplicação, e responde quando alguém pede uma pasta ou um arquivo.

1.1 O que o projeto tem

- Uma **aplicação web** feita em Python com Flask, que mostra uma interface para mexer nos arquivos.
- O **Cloudflare Tunnel**, usado através do programa Cloudflared, que permite acessar o servidor de fora da rede da casa ou da empresa.
- Uma **automação em PowerShell** (trycloudflared.ps1) que liga todos os componentes na ordem certa.
- O **NSSM**, que transforma essa automação em um serviço do Windows, para que ela comece sozinha quando o computador liga.
- Um **script Python** (enviar_email.py) que manda por e-mail o endereço gerado pelo Cloudflare Tunnel.

1.2 Por que ele foi criado

Eu queria entender, na prática, como as peças de um serviço de acesso remoto se encaixam: programação web, rede, automação e sistema operacional. Fazer um servidor de arquivos próprio obriga a mexer em todas essas áreas ao mesmo tempo, e por isso foi um bom projeto de aprendizado.

Existe também um motivo prático. O endereço que o Cloudflare gera no modo usado aqui **muda a cada vez que o túnel é iniciado**. Como o servidor liga sozinho, eu não ia saber qual era o endereço novo. Por isso foi criado o script que envia o link por e-mail. Esse detalhe vai ficar mais claro nas seções 8 e 9.

1.3 O que o projeto não é

Vale deixar claro desde já: este não é um substitutivo de serviços profissionais como Google Drive ou OneDrive. É um projeto de estudo que funciona, mas que tem limitações reais. Elas estão explicadas com calma na seção 15.

2. Tecnologias utilizadas

Antes de explicar o projeto inteiro, vale entender cada tecnologia separadamente. Para cada uma eu explico o que é, para que serve, como entra neste projeto e como ela conversa com as outras.

2.1 Python

O que é: Uma linguagem de programação. Uma linguagem de programação é a forma como a gente escreve instruções para o computador executar. O Python é conhecido por ter uma escrita mais legível que a maioria das outras linguagens.

Para que serve: Serve para criar desde pequenos scripts até sistemas completos: sites, análise de dados, automações, etc.

Como é usado neste projeto: Tudo que é lógica do sistema está em Python: o `app.py` (a aplicação web) e o `enviar_email.py` (envio do e-mail).

Relação com as outras partes: O Python roda o Flask, que é uma biblioteca feita para ele. O PowerShell chama o Python para iniciar o `app.py` e, mais tarde, o `enviar_email.py`.

```
print("Olá, servidor!")
```

2.2 Flask

O que é: Um *framework* web para Python. Framework é um conjunto de ferramentas prontas que ajuda a construir um tipo específico de programa, para a gente não precisar começar do zero. O Flask é um framework para criar sites e aplicações web.

Para que serve: Ele recebe os pedidos que chegam pelo navegador (as requisições), executa o código Python correspondente e devolve uma resposta, normalmente uma página HTML ou um arquivo.

Como é usado neste projeto: É o coração do projeto. O Flask abre a interface web, lista o conteúdo das pastas e entrega os arquivos para download. Ele fica escutando na porta 5000 do computador servidor.

Relação com as outras partes: O Flask não sabe nada sobre internet pública. Ele só atende no próprio computador. Quem leva o tráfego de fora até ele é o Cloudflare Tunnel.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Página inicial"

app.run(port=5000)
```

2.3 HTML

O que é: A linguagem que descreve o conteúdo de uma página web: títulos, textos, links, botões, formulários. O navegador lê o HTML e desenha a página na tela.

Para que serve: Define a estrutura do que o usuário vê.

Como é usado neste projeto: A interface do servidor de arquivos é uma página HTML (por exemplo, o `templates/index.html`). Ela mostra a lista de pastas e arquivos e os botões das ações.

Relação com as outras partes: O Flask entrega o HTML ao navegador. O CSS deixa esse HTML com visual agradável e o JavaScript pode dar comportamento a ele.

```
<h1>Meus arquivos</h1>
<a href="/download/relatorio.pdf">relatorio.pdf</a>
```

2.4 CSS

O que é: A linguagem que define a aparência de uma página: cores, fontes, espaçamentos, tamanho dos botões.

Para que serve: Separa o visual do conteúdo. O HTML diz *o que* existe na página e o CSS diz *como* aquilo aparece.

Como é usado neste projeto: Deixa a interface do servidor legível e organizada. Os arquivos de estilo ficam normalmente na pasta static.

Relação com as outras partes: É carregado pelo HTML e entregue pelo Flask como arquivo estático (seção 4).

```
button {
    background-color: #1F3A5F;
    color: white;
}
```

2.5 JavaScript

O que é: Uma linguagem de programação que roda dentro do navegador.

Para que serve: Dá comportamento à página: reagir a cliques, atualizar partes da tela sem recarregar, enviar arquivos em segundo plano, etc.

Como é usado neste projeto: Na interface web ele serve para as ações que acontecem na tela do usuário, como interagir com os botões. O Python roda no servidor; o JavaScript roda no navegador de quem está acessando.

Relação com as outras partes: O JavaScript conversa com o Flask fazendo requisições HTTP. O Flask responde e o JavaScript atualiza o que aparece na página.

```
document.querySelector("button").onclick = () => alert("Clicou!");
```

2.6 PowerShell

O que é: Um terminal e linguagem de script do Windows. Um script é um arquivo com uma lista de comandos que o computador executa em sequência, como se alguém estivesse digitando um por um.

Para que serve: Automatiza tarefas no Windows: abrir programas, apagar arquivos, esperar um tempo, ler a saída de outros programas.

Como é usado neste projeto: O arquivo trycloudflared.ps1 é escrito em PowerShell. Ele apaga o link antigo, inicia o Flask, inicia o Cloudflared, captura o endereço gerado, salva em link.txt e chama o script de e-mail.

Relação com as outras partes: É o "maestro" do projeto: chama o Python (Flask e e-mail) e o Cloudflared, e por sua vez é chamado pelo NSSM.

```
Write-Host "Olá do PowerShell"
```

2.7 Cloudflare Tunnel

O que é: Um serviço da Cloudflare que cria um túnel seguro entre um computador e a rede da Cloudflare. Quem acessa o endereço público da Cloudflare tem o tráfego levado por esse túnel até o seu computador.

Para que serve: Permite publicar uma aplicação que está rodando em uma máquina local sem precisar abrir portas no roteador nem ter IP fixo.

Como é usado neste projeto: É o que permite acessar o servidor Flask de fora da rede local. O projeto usa o modo que gera um endereço temporário no formato trycloudflare.com.

Relação com as outras partes: O túnel liga a internet ao Flask. Ele depende do Cloudflared (o programa) para existir no computador.

2.8 Cloudflared

O que é: O programa (executável) da Cloudflare que roda no computador do servidor. O nome vem de "Cloudflare daemon". Daemon é o nome que se dá a programas que ficam rodando em segundo plano.

Para que serve: É ele que realmente cria e mantém o túnel. O Cloudflare Tunnel é o serviço; o cloudflared é a ferramenta instalada na máquina que usa esse serviço.

Como é usado neste projeto: O trycloudflared.ps1 executa o cloudflared apontando para http://localhost:5000. O cloudflared então devolve na tela o link público temporário.

Relação com as outras partes: Recebe o tráfego vindo da Cloudflare e entrega ao Flask. O PowerShell lê o que ele escreve para descobrir o endereço.

```
cloudflared tunnel --url http://localhost:5000
```

2.9 SMTP

O que é: *Simple Mail Transfer Protocol*, o protocolo padrão para enviar e-mails pela internet. Protocolo é um conjunto de regras que dois sistemas seguem para se entender.

Para que serve: Define como um programa entrega uma mensagem a um servidor de e-mail, que depois a repassa ao destinatário.

Como é usado neste projeto: O enviar_email.py usa SMTP para conectar no servidor do Gmail, se autenticar e enviar a mensagem com o link do túnel.

Relação com as outras partes: O Python tem uma biblioteca padrão para isso. O SMTP é só o "idioma" da conversa; quem recebe é o servidor do Gmail.

2.10 Gmail

O que é: O serviço de e-mail do Google.

Para que serve: Além de caixa de entrada, ele oferece um servidor SMTP que programas podem usar para enviar mensagens.

Como é usado neste projeto: O Gmail é o servidor SMTP usado pelo script. A conta de envio é configurada em GMAIL_USUARIO e a autenticação é feita com uma **senha de aplicativo** (seção 9), que é diferente da senha normal da conta.

Relação com as outras partes: Recebe a mensagem do enviar_email.py via SMTP e entrega no endereço definido em EMAIL_DESTINO.

2.11 NSSM

O que é: O *Non-Sucking Service Manager*, uma ferramenta que permite transformar quase qualquer programa ou script em um serviço do Windows.

Para que serve: Deixa um programa rodando em segundo plano, iniciando junto com o Windows, sem precisar de ninguém logado e sem janela aberta.

Como é usado neste projeto: Neste projeto, o NSSM transforma o `trycloudflared.ps1` em um serviço chamado `servidorpython`. Ele não inicia o Flask nem o Cloudflared diretamente: ele inicia o PowerShell, e o script faz o resto (seção 11).

Relação com as outras partes: Fica entre o Windows e o PowerShell.

2.12 Windows Services

O que é: Os serviços do Windows são programas que rodam em segundo plano, gerenciados pelo próprio sistema. Exemplos: o serviço de impressão e o de atualização do Windows.

Para que serve: Permitem que algo seja iniciado automaticamente na inicialização, reiniciado se falhar e controlado por uma lista central (o gerenciador de serviços).

Como é usado neste projeto: O `servidorpython` é um serviço do Windows. Isso significa que, ao ligar o computador, o Windows já inicia a automação, sem eu precisar abrir nada.

Relação com as outras partes: O Windows é quem gerencia o serviço. O NSSM é a ponte que permite que um script comum se comporte como um serviço.

3. Arquitetura do projeto

Arquitetura é o jeito como as partes de um sistema estão organizadas e como elas conversam entre si. Aqui tem várias peças e cada uma tem uma função bem definida. Se uma delas for trocada ou falhar, é importante saber qual era o papel dela.

3.1 Quem é quem

Componente	Papel no sistema
Computador servidor	A máquina física (com Windows) onde tudo roda e onde os arquivos ficam guardados.
Armazenamento dos arquivos	A pasta no disco do servidor onde os arquivos realmente estão. É o conteúdo que o usuário quer acessar.
Aplicação Flask (app.py)	O programa que atende os pedidos, lê a pasta e entrega a interface e os arquivos.
Navegador do usuário	O programa de quem acessa (Chrome, Firefox, etc.). Pode estar em um celular ou outro computador.
Cloudflare Tunnel	O serviço da Cloudflare que recebe o acesso público e o leva até o servidor por meio de um túnel.
Cloudflared	O programa instalado no servidor que mantém esse túnel aberto.
Script PowerShell	O trycloudflared.ps1. Liga e organiza os componentes na ordem correta.
NSSM	Transforma o script PowerShell em um serviço do Windows, para iniciar automaticamente.
Script de e-mail	O enviar_email.py. Manda o link do túnel para o e-mail configurado.

3.2 Visão geral



3.3 O caminho de uma requisição

Uma requisição é um pedido feito pelo navegador. Quando alguém abre o link do servidor, é isto que acontece:

```
Usuário abre o link no navegador
    |
    v
A Cloudflare recebe a requisição
    |
    v
O túnel encaminha a requisição para o computador servidor
    |
    v
O cloudflared entrega a requisição em http://localhost:5000
    |
    v
O Flask recebe a requisição e decide o que fazer (rota)
    |
    v
O Flask acessa os arquivos na pasta de armazenamento
    |
    v
O Flask monta a resposta (página HTML ou arquivo)
    |
    v
A resposta volta pelo mesmo caminho até o navegador
```

Do ponto de vista de quem está acessando, parece que o site está hospedado na Cloudflare. Na verdade, a Cloudflare só faz o papel de intermediária. Tudo que realmente processa o pedido e guarda os dados está no meu computador.

4. Aplicação Flask

O arquivo `app.py` é a aplicação principal do projeto. É ele que transforma o computador em um servidor web: quando o `app.py` está rodando, o computador passa a ficar esperando pedidos e respondendo a eles.

Vou explicar os conceitos aos poucos, sem presumir que você já conhece Flask.

4.1 Servidor local

Um servidor, no sentido mais simples, é um programa que fica esperando pedidos e responde a eles. Quando o `app.py` roda, o Flask cria esse servidor **dentro do próprio computador**. Por isso se chama servidor local.

4.2 Localhost

localhost é um nome que significa "este mesmo computador". Quando você digita `localhost` no navegador, ele não vai para a internet: ele procura um programa rodando na sua própria máquina. É equivalente ao endereço `127.0.0.1`.

4.3 Porta 5000

Um computador pode ter vários programas de rede ao mesmo tempo. Para que eles não se misturem, cada um usa uma **porta**, que funciona como o número de um guichê em um prédio. O endereço é o prédio e a porta é o guichê.

O Flask usa por padrão a porta 5000. Então o endereço completo da aplicação, dentro do computador, é:

```
http://localhost:5000
```

4.4 Requisições e respostas HTTP

HTTP é o protocolo que o navegador e os servidores usam para conversar. Funciona em pares: o navegador manda uma **requisição** e o servidor devolve uma **resposta**.

- **Requisição:** "me dê a página inicial", "me dê este arquivo", "receba este arquivo que estou enviando".
- **Resposta:** o conteúdo pedido (uma página HTML, um arquivo) ou uma mensagem de erro, junto com um código numérico, como 200 (deu certo) ou 404 (não encontrado).

4.5 Rotas

Uma **rota** é um endereço dentro do servidor ligado a um pedaço de código. No Flask, a gente diz: "quando alguém acessar este caminho, execute esta função".

```
@app.route("/")
def inicio():
    return "Esta é a página inicial"

@app.route("/sobre")
def sobre():
    return "Esta é outra página"
```

No servidor de arquivos, as rotas são o que permite ao navegador pedir ações diferentes: listar o conteúdo de uma pasta, baixar um arquivo, enviar um arquivo, e assim por diante.

4.6 Templates

Um **template** é um HTML com "buracos" que o Flask preenche com dados antes de enviar. Em vez de escrever uma página diferente para cada pasta, existe um único modelo. O Flask coloca nele a lista de arquivos da pasta atual, e o resultado vira o HTML que o navegador recebe.

Os templates ficam na pasta `templates/`, e o principal é o `index.html`.

4.7 Arquivos estáticos

Arquivos estáticos são aqueles que não mudam conforme o pedido: arquivos CSS, arquivos JavaScript, imagens e ícones. O Flask entrega esses arquivos como são. Ficam na pasta `static/`.

4.8 Resumindo o papel do Flask

O Flask recebe a requisição, descobre qual rota corresponde, executa o código Python dessa rota (que pode ler a pasta de arquivos), usa um template para montar a página e devolve a resposta. É esse ciclo que sustenta toda a interface web.

5. Sistema de arquivos

O objetivo do projeto é mexer com arquivos que estão no disco do servidor. A aplicação usa o próprio sistema de arquivos do Windows: pastas, subpastas e arquivos normais, que continuam existindo mesmo que o Flask seja desligado.

5.1 O que a interface permite fazer

- **Navegar entre pastas:** a página mostra o conteúdo da pasta atual.
- **Entrar em diretórios:** clicar em uma pasta abre o conteúdo dela. Diretório é outro nome para pasta.
- **Voltar para diretórios anteriores:** retornar ao nível acima na hierarquia.
- **Visualizar arquivos:** ver o que existe na pasta e abrir arquivos que o navegador consegue exibir.
- **Fazer download:** baixar um arquivo do servidor para o aparelho de quem acessa.
- **Fazer upload:** enviar um arquivo do aparelho do usuário para a pasta do servidor.
- **Criar pastas:** criar novos diretórios para organizar o conteúdo.
- **Mover ou organizar arquivos:** dependendo da versão da aplicação, existem recursos para reorganizar o conteúdo entre pastas.

5.2 Por que controlar os caminhos

Quando você clica em uma pasta, o navegador manda ao servidor o **caminho** dela. Algo como fotos/viagem. O Flask junta isso com o caminho da pasta de armazenamento e vai buscar no disco.

O ponto é: **quem escolhe o caminho é o navegador**, não o servidor. E o navegador, na prática, é controlado por quem acessa. Se a aplicação aceitar qualquer caminho sem checar, ela pode acabar abrindo arquivos que nunca deveria mostrar. Por isso o controle dos caminhos é uma parte crítica, e é o assunto da próxima seção.

6. Segurança dos caminhos

6.1 O problema: Directory Traversal / Path Traversal

No sistema de arquivos, `..` significa "a pasta acima desta". Se eu estou em armazenamento/fotos, o caminho `..` leva para armazenamento. Com `../..` eu subo duas pastas.

O ataque chamado **Directory Traversal** (ou Path Traversal) explora isso. A pessoa monta um caminho cheio de `..` para sair da pasta de armazenamento e chegar em arquivos que não faziam parte do que deveria ser compartilhado.

```
Pasta de armazenamento:  C:\caminho\armazenamento
Pedido do navegador:     ..\..\outra_pasta\arquivo_pessoal.txt

Caminho montado sem checar:
C:\caminho\armazenamento\..\..\outra_pasta\arquivo_pessoal.txt
= C:\outra_pasta\arquivo_pessoal.txt    <- fora da pasta permitida!
```

Se a aplicação simplesmente juntasse as partes e entregasse o que encontrasse, qualquer pessoa com o link poderia ler arquivos do computador que não tinham nada a ver com o projeto.

6.2 A solução: validar o caminho

A solução é uma função de validação, aqui chamada de `caminho_seguro`. Antes de a aplicação usar qualquer caminho vindo do navegador, ela passa por essa função, que responde a uma pergunta: "esse caminho, de verdade, ainda está dentro da pasta de armazenamento permitida?"

Para responder, ela usa o **caminho real** (em Python, `os.path.realpath`). O `realpath` resolve todos os `..` e atalhos e devolve o caminho final de verdade. Só depois dessa conversão é possível comparar com segurança.

Segue um exemplo simplificado para mostrar a ideia (não é o código literal do projeto):

```
import os

PASTA_BASE = os.path.realpath("armazenamento")

def caminho_seguro(caminho_pedido):
    # junta a pasta permitida com o que o navegador pediu
    destino = os.path.realpath(os.path.join(PASTA_BASE, caminho_pedido))

    # confere se o caminho final continua dentro da pasta permitida
    if os.path.commonpath([PASTA_BASE, destino]) != PASTA_BASE:
        return None    # fora da pasta: recusa

    return destino    # dentro da pasta: pode usar
```

Com isso, o caminho `..\..\outra_pasta\arquivo_pessoal.txt` do exemplo anterior vira `C:\outra_pasta\...`, que não está dentro de armazenamento, e a função recusa.

Importante: Essa validação resolve um problema específico, mas **não torna o sistema completamente seguro**. Ela só garante que os caminhos não escapam da pasta permitida. Outras proteções, como autenticação de usuários, ainda são necessárias para usar o sistema de forma séria (seções 15, 16 e 19).

7. Cloudflare Tunnel

Esta é a parte que resolve o maior problema do projeto: como acessar o servidor de fora de casa ou da empresa?

7.1 Conceitos de rede

Termo	O que significa
Servidor local	Um programa rodando no seu computador esperando pedidos. Neste caso, o Flask.
localhost	O nome que significa "este computador" (127.0.0.1). Só funciona de dentro da própria máquina.
IP local	O endereço do computador dentro da rede interna (por exemplo, o formato 192.168.x.x). Dá para acessar de outros aparelhos conectados no mesmo Wi-Fi, mas não de fora.
Internet	A rede mundial. Para alguém de fora alcançar o seu computador, o pedido precisa passar pelo roteador da sua casa, que normalmente bloqueia conexões vindas de fora.
Túnel	Um caminho criado pelo próprio servidor, saindo para a internet, por onde o tráfego de fora consegue chegar até ele.

7.2 Por que localhost:5000 não funciona de fora

Quando o Flask roda e eu acesso `http://localhost:5000`, tudo acontece dentro do meu computador. Se eu mandar esse endereço para um amigo, o navegador dele vai procurar o **localhost dele**, que é o computador dele, e não vai encontrar nada.

Mesmo usando o IP local, só aparelhos na mesma rede alcançam. Para sair dessa limitação do jeito tradicional, seria preciso configurar o roteador para redirecionar portas e lidar com IP que muda. Isso é trabalhoso e deixa o computador mais exposto.

7.3 Como o Cloudflare Tunnel resolve

A ideia do túnel é inverter o sentido da conexão. Em vez de a internet tentar *entrar* na minha rede, é o meu computador que *sai* e se conecta à Cloudflare. Essa conexão de saída fica aberta (por isso "túnel"). Quando alguém acessa o endereço da Cloudflare, o pedido é levado pelo túnel até o meu computador.

```
SEM TÚNEL:
  Internet --X--> Roteador (bloqueia) --> Computador

COM TÚNEL:
  Computador =====> Cloudflare          (conexão de saída, sempre aberta)
  Usuário -----> Cloudflare =====> Computador  (pedido passa pelo túnel)
```

Assim não é preciso abrir portas no roteador nem ter IP fixo.

7.4 O papel do cloudflared

O **cloudflared** é o programa que fica rodando no servidor e mantém essa conexão com a Cloudflare. Quando chega um pedido pelo túnel, é ele quem entrega ao endereço configurado, que aqui é o Flask.

7.5 O comando utilizado

```
cloudflared tunnel --url http://localhost:5000
```

Parte	Significado
cloudflared	O programa que será executado.
tunnel	O subcomando: diz ao programa que a tarefa é criar um túnel.
--url	A opção que informa qual é o serviço local que o túnel deve expor.
http://localhost:5000	O endereço do Flask dentro do computador, ou seja, o destino final do tráfego.

7.6 O modo trycloudflare.com

Rodar o comando assim, sem criar uma conta e sem configurar domínio, usa o chamado modo rápido (*quick tunnel*). O cloudflared gera na hora um endereço no formato:

```
https://alguma-coisa-aleatoria.trycloudflare.com
```

Atenção: Esse endereço **não é um domínio permanente**. Ele é gerado para aquela execução do túnel e **pode mudar sempre que o túnel for iniciado de novo**. É por isso que o projeto tem um script que envia o link novo por e-mail a cada inicialização.

8. Script trycloudflared.ps1

O trycloudflared.ps1 é uma das partes principais da automação. Sem ele, eu teria que fazer tudo manualmente toda vez que ligasse o computador: abrir um terminal, iniciar o Flask, abrir outro terminal, iniciar o cloudflared, copiar o link e depois mandar para mim mesmo. O script faz esse caminho sozinho.

8.1 A sequência

- 1 **Apagar o link antigo.** O link.txt de uma execução anterior pode ainda existir e já estar inválido. Se ele ficasse lá, algo poderia usar um endereço velho pensando que é o atual.
- 2 **Iniciar o Flask.** Executa o app.py para a aplicação ficar disponível na porta 5000.
- 3 **Esperar alguns segundos.** O Flask leva um tempo para ficar pronto.
- 4 **Iniciar o cloudflared.** Cria o túnel apontando para localhost:5000.
- 5 **Acompanhar a saída do cloudflared.** O programa escreve mensagens (logs) enquanto trabalha. O link aparece no meio delas.
- 6 **Identificar o endereço trycloudflare.com.** O script procura, no texto da saída, algo que tenha o formato do link.
- 7 **Salvar o endereço em link.txt.** O link fica guardado em arquivo.
- 8 **Aguardar um período.** Uma folga antes de enviar.
- 9 **Executar o enviar_email.py** passando o link, para ele ser enviado por e-mail.

8.2 Por que existem esperas entre as etapas

Os programas não ficam prontos no instante em que são chamados. O PowerShell dispara um comando e já segue para o próximo, sem saber se o anterior terminou de iniciar. As esperas existem para respeitar o tempo que cada etapa leva:

- **Depois do Flask:** se o cloudflared começasse antes de o Flask estar escutando na porta 5000, o túnel ficaria apontando para um endereço onde ainda não há nada.
- **Depois do cloudflared:** o link não aparece no mesmo instante. É preciso esperar a conexão com a Cloudflare ser estabelecida. Por isso o script acompanha a saída até o endereço surgir.
- **Antes do e-mail:** enviar o e-mail antes de ter o link correto não faria sentido. Também dá um tempo para o túnel estabilizar.

8.3 Diagrama

```
Windows
  ↓
trycloudflared.ps1
  ↓
app.py
  ↓
Flask
  ↓
Cloudflared
  ↓
link.trycloudflare.com
  ↓
link.txt
  ↓
enviar_email.py
  ↓
e-mail
```

8.4 Esquema simplificado

O bloco abaixo é um esquema simplificado para mostrar a ideia, não o script real. Os tempos de espera aqui estão marcados como valores de exemplo.

```
# 1. apagar o link antigo
Remove-Item "link.txt" -ErrorAction SilentlyContinue

# 2. iniciar o Flask
Start-Process python -ArgumentList "app.py"

# 3. esperar o Flask ficar pronto
Start-Sleep -Seconds 5

# 4. iniciar o cloudflared, guardando a saída em um arquivo de log
Start-Process cloudflared `
  -ArgumentList "tunnel --url http://localhost:5000" `
  -RedirectStandardError "saida_cloudflared.txt"

# 5 e 6. acompanhar a saída até aparecer o endereço trycloudflare.com
$link = $null
while (-not $link) {
  Start-Sleep -Seconds 2
  $texto = Get-Content "saida_cloudflared.txt" -Raw
  if ($texto -match "https://[a-z0-9-]+\.\trycloudflare\.com") {
    $link = $Matches[0]
  }
}

# 7. salvar o link
Set-Content "link.txt" $link

# 8. aguardar um período
Start-Sleep -Seconds 10

# 9. enviar o e-mail
python enviar_email.py $link
```

O ponto principal para entender é o laço while: ele fica repetindo a leitura da saída do cloudflared até encontrar o endereço. É a forma de garantir que o e-mail só seja enviado depois de o link existir.

9. Script enviar_email.py

O enviar_email.py tem uma função bem específica: pegar o link gerado pelo túnel e mandar para o meu e-mail. Isso resolve o problema do endereço temporário, porque toda vez que o servidor liga e o túnel recebe um endereço novo, eu fico sabendo qual é.

9.1 Como ele recebe o link

O link é passado como **argumento**, ou seja, um valor colocado depois do nome do script na linha de comando. O trycloudflared.ps1 faz essa chamada:

```
python enviar_email.py "https://exemplo.trycloudflare.com"
```

Dentro do Python, esse valor chega em `sys.argv[1]`.

9.2 As configurações do arquivo .env

O script não tem e-mail nem senha escritos no código. Ele lê esses dados de um arquivo chamado **.env**. As variáveis usadas atualmente são:

Variável	O que representa	Exemplo fictício
GMAIL_USUARIO	A conta do Gmail usada para enviar a mensagem.	servidor.exemplo@gmail.com
GMAIL_SENHA_APP	A senha de aplicativo dessa conta, usada para o script se autenticar.	exemplo_de_senha_de_aplicativo
EMAIL_DESTINO	O endereço que vai receber o link.	usuario.exemplo@gmail.com

9.3 A senha de aplicativo

A senha de aplicativo é uma senha gerada pelo Google, separada da senha normal da conta, criada especificamente para um programa específico se conectar. Como o script não consegue fazer o login pela tela do navegador, ele usa essa senha.

Cuidado: A senha de aplicativo do Gmail deve ser tratada como uma **credencial secreta**. Quem tiver essa senha consegue enviar e-mails pela conta. Ela **nunca** deve ser colocada no código, em prints, em documentos nem publicada no GitHub.

9.4 Como funciona o envio via SMTP

O envio por SMTP é uma conversa curta entre o script e o servidor do Gmail:

- 1 O script se **conecta** ao servidor SMTP do Gmail (endereço e porta).
- 2 Ativa uma **conexão criptografada**, para ninguém no caminho conseguir ler a senha.
- 3 Faz o **login** com o usuário e a senha de aplicativo.
- 4 **Entrega a mensagem:** remetente, destinatário, assunto e texto com o link.
- 5 **Encerra** a conexão.

Exemplo simplificado (não é o código literal):

```
import os, sys, smtplib
from email.message import EmailMessage
from dotenv import load_dotenv

load_dotenv()                                # lê o arquivo .env

usuario = os.getenv("GMAIL_USUARIO")
senha   = os.getenv("GMAIL_SENHA_APP")
destino = os.getenv("EMAIL_DESTINO")
link    = sys.argv[1]                        # link recebido como argumento

msg = EmailMessage()
msg["From"]    = usuario
msg["To"]      = destino
msg["Subject"] = "Endereço do servidor"
msg.set_content("O servidor está no ar:\n" + link)

with smtplib.SMTP("smtp.gmail.com", 587) as servidor:
    servidor.starttls()                      # ativa a criptografia
    servidor.login(usuario, senha)           # autentica
    servidor.send_message(msg)               # envia
```

10. Arquivo .env

10.1 Por que não colocar dados sensíveis no código

Escrever e-mail e senha direto no código é um erro comum e perigoso. O código costuma ser copiado, versionado e compartilhado. Se a senha estiver nele, ela vai junto. E se o código for publicado em um repositório público, como o GitHub, qualquer pessoa consegue ver.

A solução é separar: o código fica de um lado e as informações sensíveis ficam em um arquivo à parte, o .env, que não é compartilhado.

10.2 Exemplo fictício

```
GMAIL_USUARIO=servidor.exemplo@gmail.com
GMAIL_SENHA_APP=exemplo_de_senha_de_aplicativo
EMAIL_DESTINO=usuario.exemplo@gmail.com
```

Cada linha tem o formato NOME=valor, sem aspas e sem espaços ao redor do sinal de igual.

10.3 O python-dotenv

O **python-dotenv** é uma biblioteca que lê o arquivo .env e coloca cada linha como uma **variável de ambiente** do programa. Variável de ambiente é um valor que fica disponível para o programa, sem estar escrito no código dele. No script, a gente chama `load_dotenv()` e depois lê com `os.getenv("NOME")`.

10.4 Por que o .env deve estar no .gitignore

O Git é a ferramenta de controle de versão usada para guardar o histórico do código, e o GitHub é o site que hospeda repositórios Git. O arquivo **.gitignore** é uma lista de arquivos que o Git deve ignorar, ou seja, nunca enviar.

Colocar o .env no .gitignore garante que, mesmo usando `git add` e `git push`, o arquivo com as credenciais não sobe para o repositório.

```
# conteúdo do .gitignore
.env
link.txt
```

Dica: Se uma senha já foi enviada ao GitHub por engano, apagar o arquivo depois não resolve, porque o histórico guarda a versão antiga. O correto é considerar a senha comprometida, revogá-la e gerar outra.

11. NSSM

11.1 O que é

NSSM significa **Non-Sucking Service Manager**. O nome é uma brincadeira, porque a ferramenta nasceu como uma alternativa mais prática às opções que já existiam para criar serviços no Windows.

11.2 Para que serve

No Windows, um programa comum precisa que alguém o abra. Um **serviço**, ao contrário, é iniciado pelo próprio sistema, em segundo plano, sem janela, e não depende de ninguém ter feito login.

O problema é que nem todo programa foi feito para ser serviço. Scripts, por exemplo, não são. O NSSM resolve isso: ele funciona como um "envelope", que o Windows sabe tratar como serviço e que, por dentro, executa o programa que a gente escolher.

11.3 Como ele foi usado neste projeto

Um ponto precisa ficar bem claro:

Diferença importante: O NSSM **NÃO** inicia diretamente o Flask nem o Cloudflared. O NSSM só transforma o script de automação **trycloudflared.ps1** em um serviço do Windows. É o script que inicia o Flask, o Cloudflared e o envio de e-mail.

```
Windows
  ↓
Serviço servidorpython
  ↓
NSSM
  ↓
PowerShell
  ↓
trycloudflared.ps1
  ↓
app.py + Cloudflared + enviar_email.py
```

Com isso, o Windows inicia o serviço sozinho na inicialização, e a automação completa começa sem que eu precise abrir o PowerShell e rodar o script.

11.4 Gerenciando o serviço

Os comandos abaixo precisam ser executados em um terminal aberto **como Administrador**. Os caminhos são genéricos; troque pelos do seu computador.

Instalar

```
nssm install servidorpython "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" ^
"-ExecutionPolicy Bypass -File C:\caminho\do\projeto\scripts\trycloudflared.ps1"
```

O primeiro valor é o programa que o serviço vai executar (o PowerShell). O segundo são os argumentos: `-File` indica qual script rodar e `-ExecutionPolicy Bypass` evita que a política de execução do Windows bloqueie o script.

Definir a pasta de trabalho (opcional, mas recomendado)

```
nssm set servidorpython AppDirectory C:\caminho\do\projeto
```

Scripts costumam usar caminhos relativos (como link.txt). Definir a pasta de trabalho faz esses caminhos funcionarem do jeito esperado.

Iniciar, parar, reiniciar e consultar

```
nssm start servidorpython  
nssm stop servidorpython  
nssm restart servidorpython  
nssm status servidorpython
```

Remover

```
nssm remove servidorpython confirm
```

O serviço também aparece na lista de serviços do Windows (services.msc), onde pode ser iniciado e parado graficamente.

12. Inicialização completa do sistema

Vamos acompanhar o que acontece desde o momento em que o botão de ligar é pressionado, na ordem:

- 1 O **Windows inicia**.
- 2 O **serviço configurado pelo NSSM** (servidorpython) é iniciado pelo sistema.
- 3 O NSSM **executa o PowerShell**.
- 4 O PowerShell **executa o trycloudflared.ps1**.
- 5 O script **inicia o Flask**.
- 6 O Flask **começa a escutar na porta 5000**.
- 7 O script **inicia o Cloudflared**.
- 8 A Cloudflare **gera o endereço temporário** trycloudflare.com.
- 9 O endereço é **salvo em link.txt**.
- 10 Depois do intervalo configurado, o **enviar_email.py é executado**.
- 11 O **e-mail é enviado**.
- 12 O usuário **recebe o endereço** e pode acessar o servidor pelo navegador.

Diagrama da sequência

```
Liga o computador
↓
Windows inicia
↓
Serviço servidorpython (NSSM) inicia
↓
NSSM executa o PowerShell
↓
PowerShell executa trycloudflared.ps1
↓
Script inicia o Flask (app.py)
↓
Flask escuta na porta 5000
↓
Script inicia o Cloudflared
↓
Cloudflare gera o link temporário
↓
Link é salvo em link.txt
↓
(intervalo configurado)
↓
enviar_email.py é executado
↓
E-mail enviado
↓
Usuário abre o link no navegador
```

13. Exemplo de uso

Para mostrar como as peças trabalham juntas, aqui vai um exemplo fictício de um dia usando o sistema.

A história

Manhã. Ana liga o computador que serve de servidor, que fica em casa. Ela não abre nenhum programa e vai tomar café.

Nos bastidores. O Windows termina de iniciar e liga o serviço servidorpython. O NSSM chama o PowerShell, que roda o trycloudflared.ps1. O script apaga o link antigo, inicia o Flask, espera alguns segundos e inicia o cloudflared. O cloudflared se conecta à Cloudflare, que gera um endereço novo. O script identifica o endereço, salva em link.txt, espera mais um pouco e chama o enviar_email.py.

Alguns minutos depois. No celular, Ana recebe um e-mail com o link, algo como `https://exemplo-aleatorio.trycloudflare.com`.

Na rua. Ana abre o link no navegador do celular. O pedido chega à Cloudflare, passa pelo túnel e entra no computador pela porta 5000. O Flask responde com a página da interface.

Navegando. Ana entra na pasta "Faculdade", depois em "Gestão de TI", encontra o PDF que precisava e clica para baixar. O Flask valida o caminho (`caminho_seguro`), lê o arquivo no disco e devolve para o celular.

O que cada parte fez

Etapa	Quem fez
Iniciar tudo sozinho	Windows + serviço do NSSM
Executar a sequência	PowerShell (trycloudflared.ps1)
Servir a interface e os arquivos	Flask (app.py)
Permitir acesso de fora	Cloudflared + Cloudflare Tunnel
Informar o link novo	enviar_email.py via SMTP (Gmail)
Guardar os arquivos	Disco do computador servidor
Impedir que o pedido saia da pasta permitida	caminho_seguro (realpath)

14. Vantagens da solução

Vou listar o que, na minha avaliação, essa solução tem de bom, sem exagerar.

- **Baixo custo.** Não há mensalidade de armazenamento. O Cloudflare Tunnel no modo rápido é gratuito.
- **Aproveitamento de hardware existente.** Um computador que talvez estivesse parado vira servidor.
- **Armazenamento local.** Os arquivos ficam no meu disco, sob o meu controle, e não em um serviço de terceiros.
- **Acesso pelo navegador.** Não precisa instalar nada no aparelho de quem acessa.
- **Automação.** Depois de configurado, o sistema liga sozinho e avisa o link por e-mail.
- **Possibilidade de expansão.** Como o projeto é modular, dá para adicionar recursos aos poucos (seção 19).
- **Aprendizado.** Em um só projeto passei por programação web, redes, automação, serviços do Windows, e-mail e noções de segurança.

Essas vantagens são reais, mas ficam dentro de um contexto: é uma solução boa para estudo, uso pessoal e testes. Ela não pretende competir com serviços profissionais de nuvem, que têm equipes inteiras cuidando de segurança, disponibilidade e backup.

15. Limitações

Esta seção é importante. Entender o que a solução não faz bem é tão necessário quanto entender o que ela faz.

Limitação	Explicação
Depende do servidor ligado	Se o computador for desligado, entrar em suspensão ou travar, os arquivos ficam inacessíveis.
Depende de internet	Sem conexão no servidor, o túnel cai. A conexão de quem acessa também influencia.
Endereço temporário	O link trycloudflare.com muda quando o túnel é reiniciado. Não dá para guardar um favorito permanente.
Segurança ainda pode melhorar	A validação de caminhos ajuda, mas é só uma camada. Há outras proteções que ainda não existem.
Sem autenticação completa de usuários	Nesta versão, não existe um sistema completo de login e permissões. Quem tiver o link pode, em princípio, acessar a interface.
Sem a infraestrutura profissional	Não há redundância, backup automático, monitoramento nem suporte, como existe em serviços de nuvem.
Desempenho limitado	A velocidade depende do hardware do servidor e, principalmente, da velocidade de <i>upload</i> da internet de onde ele está.

Nenhuma dessas limitações impede o projeto de funcionar, mas elas precisam ser conhecidas por quem for usá-lo.

16. Segurança e boas práticas

- **Nunca publicar o .env.** Ele contém as configurações sensíveis.
- **Nunca publicar senhas**, nem no código, nem em documentos, nem em prints.
- **Nunca publicar senhas de aplicativo.** Têm o mesmo risco de uma senha normal.
- **Usar o .gitignore** para o Git ignorar o .env e outros arquivos sensíveis.
- **Validar caminhos** sempre que o servidor receber um caminho do navegador.
- **Controlar quem possui o link.** Como não há login completo, o link funciona como uma chave. Compartilhe apenas com quem deve acessar.
- **Manter o sistema atualizado:** Windows, Python, Flask, cloudflared e bibliotecas.
- **Considerar autenticação** antes de usar em um ambiente real.
- **Não guardar informações extremamente sensíveis** (documentos pessoais críticos, dados financeiros, etc.) sem mecanismos adequados de proteção.

17. Estrutura do projeto

```
Servidor de armazenamento/  
├── app.py  
├── .env (não é publicado)  
├── .gitignore  
├── templates/  
│   └── index.html  
├── static/  
└── scripts/  
    ├── trycloudflared.ps1  
    └── enviar_email.py
```

Arquivo / pasta	Função
app.py	Aplicação Flask. Define as rotas, controla os arquivos e valida os caminhos.
templates/	Pasta dos modelos HTML. O index.html é a interface principal.
static/	Arquivos estáticos: CSS, JavaScript, imagens.
scripts/	Pasta dos scripts de automação.
trycloudflared.ps1	Script PowerShell que inicia e coordena Flask, Cloudflared e e-mail.
enviar_email.py	Envia o link do túnel por e-mail via SMTP.
.env	Configurações sensíveis (usuário, senha de aplicativo e destino do e-mail).
.gitignore	Lista do que o Git não deve versionar.
link.txt	Gerado durante a execução. Guarda o endereço atual do túnel.

18. Problemas encontrados durante o desenvolvimento

O projeto não funcionou de primeira. Passou por vários testes e ajustes, e os problemas que apareceram são bem comuns em projetos desse tipo. Vou comentar cada um.

Flask não iniciar corretamente

Quando o app.py tinha algum erro ou a porta 5000 estava ocupada, o Flask não subia. Sem ele, o resto da cadeia não adianta, então foi preciso garantir que ele iniciasse antes de tudo.

Cloudflared não gerar o link imediatamente

Logo que o cloudflared inicia, o link ainda não existe. Ele leva um tempo para conectar à Cloudflare. Ler o link cedo demais resultava em nada encontrado. Por isso o script acompanha a saída até o endereço aparecer.

Esperar o Flask antes de iniciar o túnel

Iniciar o túnel antes de o Flask estar escutando fazia o túnel apontar para algo que ainda não existia. A solução foi colocar uma espera entre as duas etapas.

Esperar o link antes de enviar o e-mail

O e-mail só faz sentido com o endereço correto. Se o script de e-mail rodasse antes de o link estar pronto, o e-mail sairia sem o endereço, ou com o antigo.

Problemas de autenticação SMTP

O envio pelo Gmail exige configuração correta. A senha normal da conta não é suficiente: é necessário usar a senha de aplicativo. Erros de autenticação levaram ao uso do .env com as variáveis GMAIL_USUARIO e GMAIL_SENHA_APP.

Configuração do serviço do Windows

Transformar um script em serviço tem detalhes: usar o executável correto do PowerShell, passar os argumentos certos, executar como Administrador e definir corretamente a pasta de trabalho. Um serviço roda em outro contexto de um terminal aberto manualmente, então o que funcionava na mão podia falhar como serviço.

Manter os processos funcionando em segundo plano

Como o servidor precisa rodar sem ninguém na frente do computador, os processos têm que continuar ativos sem depender de uma janela aberta. Isso foi um dos motivos para usar o NSSM.

O resultado

Todos esses problemas levaram à criação de uma **sequência automatizada**: apagar o link antigo, iniciar o Flask, esperar, iniciar o túnel, capturar o link, salvar e enviar. Cada etapa da automação existe porque, sem ela, algo deu errado em algum teste.

19. Possíveis melhorias futuras

Atenção: Tudo o que está listado nesta seção são **possibilidades futuras**. Nenhuma dessas melhorias faz parte da versão atual do projeto.

Melhoria	Para que serviria
Autenticação de usuários	Exigir login para acessar a interface.
Permissões por usuário	Definir quem pode ver, enviar ou apagar o quê.
Banco de dados	Guardar usuários, permissões e registros de forma organizada.
Logs	Registrar quem acessou e o que foi feito, para investigar problemas.
Painel administrativo	Uma tela para gerenciar usuários e configurações.
HTTPS permanente e domínio próprio	Um endereço fixo e profissional, em vez do link temporário.
Monitoramento	Avisar quando o servidor ou o túnel cair.
Limite de upload	Evitar envio de arquivos muito grandes.
Controle de espaço	Acompanhar e limitar o uso do disco.
Backup automático	Copiar os arquivos periodicamente para outro local.
Criptografia de arquivos sensíveis	Proteger o conteúdo mesmo em caso de acesso indevido ao disco.
Testes automatizados	Verificar se as funções continuam corretas depois de mudanças no código.

20. Conclusão

Este projeto começou com uma ideia simples, usar um computador comum como servidor de arquivos, e mostrou ser mais do que um servidor Flask simples. Para funcionar bem, foi preciso juntar várias áreas:

- **Desenvolvimento web:** aplicação Flask com rotas, templates e arquivos estáticos.
- **Gerenciamento de arquivos:** navegar, baixar, enviar e organizar conteúdo no disco.
- **Automação:** um script PowerShell que coordena a inicialização na ordem certa.
- **Redes:** localhost, portas, IP local e a diferença entre rede interna e internet.
- **Acesso remoto:** Cloudflare Tunnel e cloudflared, que dispensam abrir portas.
- **Integração com serviços externos:** Cloudflare e Gmail.
- **SMTP:** envio automático do link por e-mail.
- **Serviços do Windows:** NSSM para iniciar a automação junto com o sistema.
- **Noções básicas de segurança:** validação de caminhos, .env, .gitignore.

O resultado é uma solução funcional de armazenamento e acesso remoto: o computador liga, a automação inicia sozinha, o túnel é criado, o link chega por e-mail e os arquivos podem ser acessados pelo navegador.

Também ficou claro, ao longo do trabalho, onde estão os limites da solução. Não existe autenticação completa, o endereço é temporário e o sistema depende de um único computador. Conhecer esses pontos é parte do que o projeto ensinou, e eles apontam o caminho das próximas melhorias.